

SM3Det 论文深度解读与课题改进方案

第一部分：SM3Det 论文核心解读

1.1 核心创新点

1.1.1 任务定义：M2Det 论文提出了 **Multi-Modal Datasets and Multi-Task Object Detection (M2Det)** 任务，目标是：

- 使用**统一模型**检测来自**任意模态**的图像中的目标
- 处理**多种预定义检测任务**（水平框 HBB、旋转框 OBB）
- 无需空间对齐的图像对

1.1.2 架构创新 A. Grid-Level Sparse MoE Backbone

核心公式：

$$f_{\text{MoE}}(x_{ij}) = \sum_{n=1}^N G_n(x_{ij}) \cdot \text{Conv1}\times\text{1}_n(x_{ij})$$

$$G(x_{ij}) = \text{Top}_k(\text{Softmax}((E^T W x_{ij}) / (\tau ||Wx_{ij}|| ||E||)))$$

关键设计：

- 网格级别路由**：在特征图的空间网格上动态选择专家（而非图像级别）
- 稀疏激活**：Top-k 机制（k=2）只激活部分专家，降低计算复杂度
- 模态特定表示**：不同专家学习不同模态的特定模式
- 共享表示学习**：部分专家跨模态共享知识

B. Dynamic Submodule Optimization (DSO)

核心机制：

1. 任务头学习率调整（基于收敛速度）：

$$w_t^i = \text{his_L}_t^i / \text{cur_L}_t^i$$

$$\lambda_t^i = \exp(w_t^i / \theta) / \sum_k \exp(w_k^i / \theta)$$

2. Backbone 学习率调整（基于一致性分数）：

$$C^i = 1 - D_{\text{KL}}(P(\text{cur_L}^i) || P(\text{his_L}^i))$$

$$\gamma^i = 2 \cdot \text{Sigmoid}((C^i - b) \cdot \tau) / (1 + \exp(-(C^i - b) \cdot \tau))$$

关键设计：

- 任务头 LR 调整**：收敛快的任务降低学习率，收敛慢的提高学习率
- Backbone LR 调整**：根据历史一致性动态调整，保持优化方向一致
- 双策略协同**：同时平衡收敛速度和优化方向

1.2 热门标签与原创性分析

概念/模块	类型	说明
M2Det 任务	原创定义	明确定义多模态 + 多任务联合检测任务

Table 1 – continued

概念/模块	类型	说明
Grid-Level MoE	原创改进	将 MoE 从图像级别下放到网格级别，适配检测任务
DSO 优化器	原创设计	首次提出基于一致性的动态学习率调整机制
Sparse MoE	已有技术	引用 Shazeer et al. 2017 的稀疏门控 MoE
Multi-Task Learning	成熟领域	引用 GradNorm, Uncertainty Loss 等已有方法
Multi-Dataset Detection	已有研究	引用 UniDet, DA Network 等
Remote Sensing	应用领域	结合 SARDet-100K, DOTA, DroneVehicle 数据集

评价：

- 真正原创：Grid-Level MoE 的路由粒度设计、DSO 的双策略动态优化机制
- 渲染性质：M2Det 任务定义（概念包装）、“低空空域经济”等应用描述
- 直接引用：Sparse MoE 基础架构、多任务学习损失加权方法

1.3 训练与硬件配置细节

```
# 硬件配置
GPUs: 8 × RTX 3090
Batch Size: 4 per GPU (total 32)

# 优化器配置
Optimizer: AdamW
Initial Learning Rate: 0.0001
Weight Decay: 0.05
Epochs: 12

# 数据采样策略
Dataset Ratio (SARDet:DOTA:DroneVehicle): 2:1:1
Cycle Frequency: ~20K iterations per dataset cycle

# MoE 配置
Number of Experts (N): 8
Top-k Selection: 2
Expert Position: Even-indexed layers of last 3 stages

# DSO 超参数
Temperature (τ): 3
Bias (b): 0.4
EMA Smoothing Factor (α): 0.1 (implied)

# FLOPs 计算
```

Input Size: 800×800
Baseline (ConvNext-T): 403G FLOPs, 66M params
SM3Det (with MoE): 487G FLOPs, 178M params

第二部分：课题改进方案设计

2.1 你的核心观察总结

目标类型	尺寸特征	特征密度	处理挑战
船只	小尺寸	纹理密集	细节保留
尾迹	大尺寸	特征稀疏	上下文建模
几何关系	位置关联	可互为先验	跨目标注意力

2.2 你的构想评价与细节补充

创新点 1: Grid-Level MoE → 单模态双目标特化 你的构想：将多模态 MoE 改为针对单模态不同特征目标的检测

评价:  方向正确

补充细节:

```
# 建议的 Expert 分工策略
experts_config = {
    "expert_0_1": "船只专用 - 高分辨率细节提取", # 使用 3x3 卷积
    "expert_2_3": "尾迹专用 - 大范围上下文聚合", # 使用条带卷积/空洞卷积
    "expert_4_5": "通用特征 - 共享表示学习", # 标准 1x1 卷积
    "expert_6_7": "边界融合 - 船-尾迹过渡区域" # 边缘感知卷积
}
```

路由策略建议:

- 基于特征统计（如局部方差、梯度幅值）自动路由
- 高方差区域 → 船只专家
- 低方差但大范围相关 → 尾迹专家

创新点 2: 交错残差连接设计 你的构想：残差连接与 MoE 并行，交错使用条带卷积和低频滤波

评价:  非常有创意，但需要精细化设计

补充细节:

```
# 4 层 Backbone 的交错变换设计
residual_transforms = {
    # Layer 1: 高分辨率, 优先处理船只细节
    "layer1": {
        "transform": "StripConv",
        "config": {
            "kernel_size": (1, 7), # 水平条带, 捕捉船只横向纹理
            "dilation": 1,
            "groups": 4 # 分组减少计算
        }
    }
```

```
    },
    "rationale": "船只在早期层有强响应，条带卷积匹配船只长宽比"
  },

  # Layer 2: 中等分辨率，开始引入尾迹低频信息
  "layer2": {
    "transform": "LowFreqFilter",
    "config": {
      "type": "gaussian_blur",
      "kernel_size": 5,
      "sigma": 2.0,
      "residual_ratio": 0.5 # 低频 + 原始特征混合
    },
    "rationale": "尾迹需要平滑的上下文，高斯滤波抑制高频噪声"
  },

  # Layer 3: 低分辨率，船只尾迹融合
  "layer3": {
    "transform": "HybridConv",
    "config": {
      "branches": [
        {"type": "strip", "kernel": (7, 1)}, # 垂直条带
        {"type": "depthwise", "kernel": 5} # 局部细节
      ],
      "fusion": "attention"
    },
    "rationale": "船尾几何关系需要多方向特征"
  },

  # Layer 4: 最低分辨率，全局上下文
  "layer4": {
    "transform": "DilatedConv",
    "config": {
      "kernel_size": 3,
      "dilation": 4,
      "padding": 4
    },
    "rationale": "最大感受野捕获全局尾迹模式"
  }
}
```

关键技术选择建议:

变换类型	适用层	适用目标	实现方式
条带卷积	Layer 1, 3	船只	nn.Conv2d(kernel_size=(1, k) or (k, 1))
高斯滤波	Layer 2	尾迹	cv2.GaussianBlur 或可学习参数
空洞卷积	Layer 4	全局	nn.Conv2d(dilation=d)
深度可分离	All	效率	Depthwise + Pointwise

创新点 3: Dual Stream 管线 + 对称注意力蒙版 你的构想: 每层对称生成注意力蒙版, 互相引导

评价: ☒ **核心创新点**, 但需要明确实现细节

补充细节:

```
# Dual Stream 架构设计
class DualStreamBlock(nn.Module):
    """
    双流块: 船只流和尾迹流, 互相通过注意力蒙版引导
    """
    def __init__(self, channels):
        super().__init__()
        # 主处理流
        self.ship_stream = MoELayer(channels, num_experts=4)
        self.wake_stream = MoELayer(channels, num_experts=4)

        # 蒙版生成器
        self.ship_mask_gen = MaskGenerator(channels) # 输出: [B, 1, H, W]
        self.wake_mask_gen = MaskGenerator(channels)

        # 交叉引导卷积
        self.cross_guidance = CrossGuidanceConv(channels)

    def forward(self, x, prev_ship_mask=None, prev_wake_mask=None):
        B, C, H, W = x.shape

        # 1. 生成当前层蒙版
        ship_mask = self.ship_mask_gen(x) # [B, 1, H, W], sigmoid 输出
        wake_mask = self.wake_mask_gen(x)

        # 2. 如果有上一层的蒙版, 用于引导当前层
        if prev_ship_mask is not None:
            # 上采样到当前分辨率
            prev_ship_mask = F.interpolate(prev_ship_mask, size=(H, W), mode='bilinear')
            # 尾迹流受船只蒙版引导 (船只位置指示尾迹可能存在区域)
            x_wake_guided = x * (1 + prev_ship_mask) # 增强船只区域
        else:
            x_wake_guided = x

        if prev_wake_mask is not None:
            prev_wake_mask = F.interpolate(prev_wake_mask, size=(H, W), mode='bilinear')
            # 船只流受尾迹蒙版引导 (尾迹区域可能包含船只)
            x_ship_guided = x * (1 + prev_wake_mask)
        else:
            x_ship_guided = x

        # 3. 双流处理
        ship_features = self.ship_stream(x_ship_guided)
        wake_features = self.wake_stream(x_wake_guided)

        # 4. 交叉融合
```

```
fused = self.cross_guidance(ship_features, wake_features, ship_mask, wake_mask)

return fused, ship_mask, wake_mask

class MaskGenerator(nn.Module):
    """ 轻量级蒙版生成器 """
    def __init__(self, channels):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(channels, channels // 4, 3, padding=1),
            nn.BatchNorm2d(channels // 4),
            nn.ReLU(inplace=True),
            nn.Conv2d(channels // 4, 1, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.conv(x)
```

蒙版引导策略:

```
# 几何先验: 船只-尾迹空间关系建模
def apply_geometric_prior(ship_mask, wake_mask, direction_prior='behind'):
    """
    利用船只和尾迹的几何关系增强蒙版
    假设: 尾迹通常在船只后方
    """
    if direction_prior == 'behind':
        # 对船只蒙版进行偏移, 预测尾迹区域
        kernel = torch.tensor([[0, 0, 0],
                                [0, 0, 0],
                                [1, 1, 1]]) # 向下偏移 (假设船向上行驶)
        ship_shifted = F.conv2d(ship_mask, kernel.view(1, 1, 3, 3).to(ship_mask.device),
                                padding=1)
        # 融合预测区域和原始尾迹蒙版
        wake_mask_enhanced = torch.max(wake_mask, ship_shifted)
        return ship_mask, wake_mask_enhanced

    return ship_mask, wake_mask
```

2.3 与 SM3Det 的差异总结

维度	SM3Det	你的方案
模态	多模态 (SAR, Optical, IR)	单模态 (Optical)
目标类型	多类别 (Ship, Aircraft, Car...)	2 类 (船只, 尾迹)
任务	多任务 (HBB + OBB)	单任务 (OBB only)
MoE 设计	模态级别专家分工	目标特征级别专家分工
残差连接	标准残差	交错变换残差
跨目标交互	无	Dual Stream + 蒙版引导
DSO	跨模态 + 跨任务	跨目标类型优化

第三部分：Kimi Code 编程指导报告

3.1 项目结构建议

```
WakeShipDet/
├── configs/
│   ├── base_config.py          # 基础配置
│   ├── wakeship_r50_fpn.py     # ResNet50 配置
│   └── wakeship_convnext_t.py  # ConvNext-T 配置
├── models/
│   ├── __init__.py
│   ├── wakeship_det.py         # 主模型入口
│   ├── backbone/
│   │   ├── __init__.py
│   │   ├── dual_stream_backbone.py # 双流 backbone
│   │   ├── moe_layer.py         # Grid-Level MoE
│   │   └── residual_transforms.py # 交错残差变换
│   ├── neck/
│   │   └── fpn.py               # 特征金字塔
│   └── head/
│       ├── ship_head.py        # 船只检测头
│       ├── wake_head.py        # 尾迹检测头
│       └── dso_optimizer.py     # DSO 优化器
├── datasets/
│   ├── __init__.py
│   ├── wakeship_dataset.py     # Swimship 数据集适配
│   └── transforms.py           # 数据增强
├── utils/
│   ├── mask_visualizer.py      # 蒙版可视化工具
│   ├── checkpoint.py           # 检查点管理
│   └── logger.py               # 日志记录
├── tools/
│   ├── train.py                # 训练脚本
│   ├── test.py                 # 测试脚本
│   └── visualize_masks.py      # 蒙版可视化脚本
├── requirements.txt
└── README.md
```

3.2 核心模块实现指导

3.2.1 Grid-Level MoE (单模态双目标版本)

```
# models/backbone/moe_layer.py
import torch
import torch.nn as nn
import torch.nn.functional as F

class GridLevelMoE(nn.Module):
    """
    针对单模态双目标检测的 Grid-Level MoE
```

```

Args:
    in_channels: 输入通道数
    num_experts: 专家数量 (建议 8)
    top_k: 激活专家数 (建议 2)
    expert_type: 专家类型分工
"""
def __init__(self, in_channels, num_experts=8, top_k=2):
    super().__init__()
    self.num_experts = num_experts
    self.top_k = top_k
    self.temperature = 1.0 # 可学习或固定

    # 专家网络 - 不同类型专家使用不同卷积配置
    self.experts = nn.ModuleList([
        self._create_expert(in_channels, expert_id=i)
        for i in range(num_experts)
    ])

    # 门控网络
    self.gate_conv = nn.Conv2d(in_channels, num_experts, 1)

    # 专家嵌入 (用于相似度计算)
    self.expert_embeddings = nn.Parameter(torch.randn(num_experts, in_channels))

def _create_expert(self, channels, expert_id):
    """ 根据专家 ID 创建不同类型的专家 """
    if expert_id in [0, 1]:
        # 船只专家: 标准 3x3 卷积, 捕获细节
        return nn.Conv2d(channels, channels, 3, padding=1, groups=4)
    elif expert_id in [2, 3]:
        # 尾迹专家: 条带卷积, 捕获大范围模式
        return nn.Conv2d(channels, channels, (1, 7), padding=(0, 3))
    elif expert_id in [4, 5]:
        # 通用专家: 1x1 卷积, 通道混合
        return nn.Conv2d(channels, channels, 1)
    else:
        # 边界专家: 深度可分离卷积
        return nn.Sequential(
            nn.Conv2d(channels, channels, 3, padding=1, groups=channels),
            nn.Conv2d(channels, channels, 1)
        )

def forward(self, x):
    B, C, H, W = x.shape

    # 计算门控分数 [B, num_experts, H, W]
    gate_logits = self.gate_conv(x)

    # Top-k 选择
    top_k_logits, top_k_indices = torch.topk(gate_logits, self.top_k, dim=1)

```



```

top_k_gates = F.softmax(top_k_logits / self.temperature, dim=1)

# 初始化输出
output = torch.zeros_like(x)

# 收集所有位置的 top-k 专家索引
for b in range(B):
    for h in range(H):
        for w in range(W):
            for k in range(self.top_k):
                expert_idx = top_k_indices[b, k, h, w]
                gate_val = top_k_gates[b, k, h, w]

                # 提取局部特征并送入对应专家
                local_feat = x[b:b+1, :, h:h+1, w:w+1]
                expert_out = self.experts[expert_idx](local_feat)
                output[b:b+1, :, h:h+1, w:w+1] += gate_val * expert_out

return output

```

3.2.2 交错残差变换模块

```

# models/backbone/residual_transforms.py
import torch
import torch.nn as nn
import torch.nn.functional as F

class StripConv(nn.Module):
    """ 条带卷积 - 适用于船只特征 """
    def __init__(self, channels, kernel_size=(1, 7), dilation=1):
        super().__init__()
        padding = (0, (kernel_size[1] - 1) // 2 * dilation)
        self.conv = nn.Conv2d(channels, channels, kernel_size,
                               padding=padding, dilation=dilation, groups=4)
        self.bn = nn.BatchNorm2d(channels)
        self.relu = nn.ReLU(inplace=True)

    def forward(self, x):
        return self.relu(self.bn(self.conv(x)))

class LowFreqFilter(nn.Module):
    """ 低频滤波 - 适用于尾迹特征 """
    def __init__(self, channels, kernel_size=5, sigma=2.0):
        super().__init__()
        self.kernel_size = kernel_size
        self.sigma = sigma
        self.residual_ratio = nn.Parameter(torch.tensor(0.5))

        # 可学习的高斯核
        self.gaussian_weight = nn.Parameter(
            self._create_gaussian_kernel(kernel_size, sigma).view(1, 1, kernel_size, kernel_size)

```

```

    )

    def _create_gaussian_kernel(self, size, sigma):
        """ 创建高斯核 """
        coords = torch.arange(size, dtype=torch.float32) - (size - 1) / 2
        g = torch.exp(-(coords ** 2) / (2 * sigma ** 2))
        g = g / g.sum()
        kernel = g.view(1, -1) * g.view(-1, 1)
        return kernel

    def forward(self, x):
        B, C, H, W = x.shape
        # 应用高斯滤波
        gaussian_kernel = self.gaussian_weight.repeat(C, 1, 1, 1).to(x.device)
        filtered = F.conv2d(x, gaussian_kernel, padding=self.kernel_size//2, groups=C)
        # 残差混合
        return self.residual_ratio * filtered + (1 - self.residual_ratio) * x

class InterleavedResidualBlock(nn.Module):
    """ 交错残差块 - 根据层数选择不同变换 """
    def __init__(self, channels, layer_idx):
        super().__init__()
        self.layer_idx = layer_idx

        # 根据层索引选择变换类型
        if layer_idx == 0:
            self.transform = StripConv(channels, kernel_size=(1, 7))
        elif layer_idx == 1:
            self.transform = LowFreqFilter(channels, kernel_size=5)
        elif layer_idx == 2:
            self.transform = nn.Sequential(
                StripConv(channels, kernel_size=(7, 1)),
                nn.Conv2d(channels, channels, 3, padding=1, groups=channels)
            )
        else: # layer_idx == 3
            self.transform = nn.Conv2d(channels, channels, 3, padding=4, dilation=4)

    def forward(self, x):
        residual = self.transform(x)
        return x + residual

```

3.2.3 Dual Stream Backbone

```

# models/backbone/dual_stream_backbone.py
import torch
import torch.nn as nn
import torch.nn.functional as F

class DualStreamBackbone(nn.Module):
    """
    双流 Backbone: 船只流和尾迹流, 互相通过蒙版引导

```

```

"""

def __init__(self, base_backbone='convnext_tiny', num_stages=4):
    super().__init__()
    self.num_stages = num_stages

    # 加载预训练 backbone 作为基础
    self.base_backbone = self._load_backbone(base_backbone)

    # 为每个 stage 创建双流模块
    self.dual_stream_blocks = nn.ModuleList([
        DualStreamBlock(self._get_stage_channels(base_backbone, i))
        for i in range(num_stages)
    ])

    # 蒙版保存 (用于可视化)
    self.saved_masks = {}

def _load_backbone(self, name):
    """ 加载基础 backbone """
    if 'convnext' in name:
        import timm
        return timm.create_model(name, pretrained=True, features_only=True)
    # 其他 backbone...

def forward(self, x, return_masks=False):
    """
    Args:
        x: 输入图像 [B, 3, H, W]
        return_masks: 是否返回中间蒙版用于可视化
    Returns:
        features: 各 stage 特征列表
        masks: 各 stage 的船只/尾迹蒙版 (如果 return_masks=True)
    """
    features = []
    masks = []

    prev_ship_mask = None
    prev_wake_mask = None

    for stage_idx in range(self.num_stages):
        # 基础特征提取
        x = self.base_backbone.stages[stage_idx](x)

        # 双流处理
        x, ship_mask, wake_mask = self.dual_stream_blocks[stage_idx](
            x, prev_ship_mask, prev_wake_mask
        )

        features.append(x)
        masks.append({'ship': ship_mask, 'wake': wake_mask})

```

```

        # 保存蒙版供下一层使用
        prev_ship_mask = ship_mask
        prev_wake_mask = wake_mask

        # 保存用于可视化
        self.saved_masks[f'stage_{stage_idx}'] = {
            'ship': ship_mask.detach(),
            'wake': wake_mask.detach()
        }

    if return_masks:
        return features, masks
    return features

class DualStreamBlock(nn.Module):
    """ 双流块：处理船只和尾迹特征 """
    def __init__(self, channels):
        super().__init__()
        # MoE 层
        from .moe_layer import GridLevelMoE
        self.ship_moe = GridLevelMoE(channels, num_experts=4, top_k=2)
        self.wake_moe = GridLevelMoE(channels, num_experts=4, top_k=2)

        # 蒙版生成器
        self.ship_mask_gen = nn.Sequential(
            nn.Conv2d(channels, channels // 4, 3, padding=1),
            nn.BatchNorm2d(channels // 4),
            nn.ReLU(inplace=True),
            nn.Conv2d(channels // 4, 1, 1),
            nn.Sigmoid()
        )

        self.wake_mask_gen = nn.Sequential(
            nn.Conv2d(channels, channels // 4, 3, padding=1),
            nn.BatchNorm2d(channels // 4),
            nn.ReLU(inplace=True),
            nn.Conv2d(channels // 4, 1, 1),
            nn.Sigmoid()
        )

        # 特征融合
        self.fusion = nn.Conv2d(channels * 2, channels, 1)

    def forward(self, x, prev_ship_mask=None, prev_wake_mask=None):
        B, C, H, W = x.shape

        # 生成当前层蒙版
        ship_mask = self.ship_mask_gen(x)
        wake_mask = self.wake_mask_gen(x)

        # 应用上一层蒙版引导

```

```

if prev_ship_mask is not None:
    prev_ship_mask = F.interpolate(prev_ship_mask, size=(H, W), mode='bilinear')
    # 尾迹流受船只蒙版引导
    wake_input = x * (0.5 + 0.5 * prev_ship_mask)
else:
    wake_input = x

if prev_wake_mask is not None:
    prev_wake_mask = F.interpolate(prev_wake_mask, size=(H, W), mode='bilinear')
    # 船只流受尾迹蒙版引导
    ship_input = x * (0.5 + 0.5 * prev_wake_mask)
else:
    ship_input = x

# 双流处理
ship_feat = self.ship_moe(ship_input)
wake_feat = self.wake_moe(wake_input)

# 融合
concat_feat = torch.cat([ship_feat, wake_feat], dim=1)
output = self.fusion(concat_feat)

return output, ship_mask, wake_mask

```

3.2.4 DSO 优化器（双目标版本）

```

# models/head/dso_optimizer.py
import torch
import torch.nn as nn

class DS0Optimizer:
    """
    针对船只-尾迹双目标检测的 DSO 优化器
    """
    def __init__(self, model, cfg):
        self.model = model
        self.cfg = cfg

        # 超参数
        self.temperature = cfg.get('dso_temperature', 3.0)
        self.bias = cfg.get('dso_bias', 0.4)
        self.ema_alpha = cfg.get('dso_ema_alpha', 0.1)

        # 历史损失 (EMA)
        self.historical_losses = {
            'ship': 1.0,
            'wake': 1.0
        }

        # 优化器
        self.optimizer = self._create_optimizer()

```

```

def _create_optimizer(self):
    """ 创建带 DSO 的优化器 """
    # 分离 backbone 和 head 参数
    backbone_params = []
    ship_head_params = []
    wake_head_params = []

    for name, param in self.model.named_parameters():
        if not param.requires_grad:
            continue
        if 'ship_head' in name:
            ship_head_params.append(param)
        elif 'wake_head' in name:
            wake_head_params.append(param)
        else:
            backbone_params.append(param)

    return torch.optim.AdamW([
        {'params': backbone_params, 'lr': self.cfg['lr']},
        {'params': ship_head_params, 'lr': self.cfg['lr']},
        {'params': wake_head_params, 'lr': self.cfg['lr']}
    ], weight_decay=self.cfg['weight_decay'])

def step(self, losses):
    """
    Args:
        losses: dict with 'ship' and 'wake' loss values
    """
    # 更新历史损失 EMA
    for task in ['ship', 'wake']:
        cur_loss = losses[task].item()
        self.historical_losses[task] = (
            self.ema_alpha * cur_loss +
            (1 - self.ema_alpha) * self.historical_losses[task]
        )

    # 计算任务头学习率调整
    ship_w = self.historical_losses['ship'] / losses['ship'].item()
    wake_w = self.historical_losses['wake'] / losses['wake'].item()

    # Softmax 归一化
    exp_ship = torch.exp(torch.tensor(ship_w / self.temperature))
    exp_wake = torch.exp(torch.tensor(wake_w / self.temperature))
    sum_exp = exp_ship + exp_wake

    ship_lambda = (exp_ship / sum_exp).item()
    wake_lambda = (exp_wake / sum_exp).item()

    # 调整任务头学习率
    self.optimizer.param_groups[1]['lr'] = self.cfg['lr'] * ship_lambda

```

```

self.optimizer.param_groups[2]['lr'] = self.cfg['lr'] * wake_lambda

# 计算 backbone 一致性分数
cur_losses = torch.tensor([losses['ship'].item(), losses['wake'].item()])
his_losses = torch.tensor([self.historical_losses['ship'],
                           self.historical_losses['wake']])

# KL 散度计算一致性
p_cur = torch.softmax(cur_losses, dim=0)
p_his = torch.softmax(his_losses, dim=0)
kl_div = torch.sum(p_cur * torch.log(p_cur / (p_his + 1e-8) + 1e-8))
consistency = 1 - kl_div.item()

# Sigmoid 调整 backbone 学习率
gamma = 2 / (1 + torch.exp(-(consistency - self.bias) * self.temperature))
self.optimizer.param_groups[0]['lr'] = self.cfg['lr'] * gamma.item()

# 反向传播
total_loss = ship_lambda * losses['ship'] + wake_lambda * losses['wake']
self.optimizer.zero_grad()
total_loss.backward()
self.optimizer.step()

return {
    'total_loss': total_loss.item(),
    'ship_lambda': ship_lambda,
    'wake_lambda': wake_lambda,
    'backbone_gamma': gamma.item()
}

```

3.2.5 蒙版可视化工具

```

# utils/mask_visualizer.py
import torch
import numpy as np
import matplotlib.pyplot as plt
import cv2

def visualize_masks(image, masks, save_path=None):
    """
    可视化 Dual Stream 生成的注意力蒙版

    Args:
        image: 原始图像 [3, H, W] 或 [H, W, 3]
        masks: dict with 'stage_0', 'stage_1', ... each containing 'ship' and 'wake'
        save_path: 保存路径
    """
    num_stages = len(masks)
    fig, axes = plt.subplots(num_stages, 3, figsize=(12, 4 * num_stages))

    if num_stages == 1:

```

```

axes = axes.reshape(1, -1)

# 处理输入图像
if isinstance(image, torch.Tensor):
    image = image.cpu().numpy()
if image.shape[0] == 3:
    image = image.transpose(1, 2, 0)

for stage_idx in range(num_stages):
    stage_key = f'stage_{stage_idx}'
    if stage_key not in masks:
        continue

    ship_mask = masks[stage_key]['ship']
    wake_mask = masks[stage_key]['wake']

    if isinstance(ship_mask, torch.Tensor):
        ship_mask = ship_mask.cpu().numpy()
    if isinstance(wake_mask, torch.Tensor):
        wake_mask = wake_mask.cpu().numpy()

# 去除 batch 维度
if ship_mask.ndim == 4:
    ship_mask = ship_mask[0, 0]
elif ship_mask.ndim == 3:
    ship_mask = ship_mask[0]

if wake_mask.ndim == 4:
    wake_mask = wake_mask[0, 0]
elif wake_mask.ndim == 3:
    wake_mask = wake_mask[0]

# 上采样到图像尺寸
h, w = image.shape[:2]
ship_mask = cv2.resize(ship_mask, (w, h))
wake_mask = cv2.resize(wake_mask, (w, h))

# 叠加显示
ship_overlay = image.copy()
wake_overlay = image.copy()

# 船只蒙版用红色叠加
ship_color = np.zeros_like(image)
ship_color[:, :, 0] = ship_mask * 255 # Red channel
ship_overlay = cv2.addWeighted(ship_overlay, 0.7, ship_color, 0.3, 0)

# 尾迹蒙版用蓝色叠加
wake_color = np.zeros_like(image)
wake_color[:, :, 2] = wake_mask * 255 # Blue channel
wake_overlay = cv2.addWeighted(wake_overlay, 0.7, wake_color, 0.3, 0)

```



```

# 绘制
axes[stage_idx, 0].imshow(image)
axes[stage_idx, 0].set_title(f'Original Image')
axes[stage_idx, 0].axis('off')

axes[stage_idx, 1].imshow(ship_overlay)
axes[stage_idx, 1].set_title(f'Stage {stage_idx} - Ship Mask')
axes[stage_idx, 1].axis('off')

axes[stage_idx, 2].imshow(wake_overlay)
axes[stage_idx, 2].set_title(f'Stage {stage_idx} - Wake Mask')
axes[stage_idx, 2].axis('off')

plt.tight_layout()

if save_path:
    plt.savefig(save_path, dpi=150, bbox_inches='tight')
    print(f"Saved visualization to {save_path}")
else:
    plt.show()

plt.close()

def save_intermediate_outputs(model, image, output_dir):
    """
    保存模型中间输出用于调试
    """
    import os
    os.makedirs(output_dir, exist_ok=True)

    model.eval()
    with torch.no_grad():
        _, masks = model(image.unsqueeze(0), return_masks=True)

    # 可视化并保存
    visualize_masks(image, model.saved_masks,
                    save_path=os.path.join(output_dir, 'masks.png'))

    return masks

```

3.3 数据集适配

```

# datasets/wakeship_dataset.py
import torch
from torch.utils.data import Dataset
import os
import cv2
import numpy as np
from pycocotools.coco import COCO

class SwimshipDataset(Dataset):

```

```

"""
Swimship 数据集适配器
支持船只和尾迹的 OBB 标注
"""

def __init__(self, root_dir, annotation_file, transforms=None):
    self.root_dir = root_dir
    self.coco = COCO(annotation_file)
    self.image_ids = self.coco.getImgIds()
    self.transforms = transforms

    # 类别映射
    self.cat_ids = self.coco.getCatIds()
    self.cat_to_idx = {cat_id: idx for idx, cat_id in enumerate(self.cat_ids)}

def __len__(self):
    return len(self.image_ids)

def __getitem__(self, idx):
    img_id = self.image_ids[idx]
    img_info = self.coco.loadImgs(img_id)[0]

    # 读取图像
    img_path = os.path.join(self.root_dir, img_info['file_name'])
    image = cv2.imread(img_path)
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # 获取标注
    ann_ids = self.coco.getAnnIds(imgIds=img_id)
    anns = self.coco.loadAnns(ann_ids)

    # 解析 OBB 标注
    bboxes = []
    labels = []

    for ann in anns:
        # COCO 格式: [x, y, width, height, angle] 或其他 OBB 格式
        # 根据实际数据集格式调整
        if 'segmentation' in ann:
            # 从分割掩码计算 OBB
            seg = ann['segmentation'][0]
            pts = np.array(seg).reshape(-1, 2)
            rect = cv2.minAreaRect(pts.astype(np.float32))
            # rect: ((cx, cy), (w, h), angle)
            bbox = [rect[0][0], rect[0][1], rect[1][0], rect[1][1], rect[2]]
        else:
            bbox = ann['bbox'] # 假设已经是 OBB 格式

        bboxes.append(bbox)
        labels.append(self.cat_to_idx[ann['category_id']])

    bboxes = torch.tensor(bboxes, dtype=torch.float32)

```

```

labels = torch.tensor(labels, dtype=torch.long)

# 应用变换
if self.transforms:
    transformed = self.transforms(image=image, bboxes=bboxes, labels=labels)
    image = transformed['image']
    bboxes = torch.tensor(transformed['bboxes'])
    labels = torch.tensor(transformed['labels'])
else:
    image = torch.from_numpy(image).permute(2, 0, 1).float() / 255.0

target = {
    'bboxes': bboxes,
    'labels': labels,
    'image_id': torch.tensor([img_id])
}

return image, target

```

3.4 训练脚本框架

```

# tools/train.py
import torch
import argparse
from tqdm import tqdm

def train_epoch(model, dataloader, dso_optimizer, device, epoch):
    model.train()
    total_loss = 0

    pbar = tqdm(dataloader, desc=f'Epoch {epoch}')
    for images, targets in pbar:
        images = images.to(device)
        targets = {k: v.to(device) if isinstance(v, torch.Tensor) else v
                    for k, v in targets.items()}

        # 前向传播
        losses = model(images, targets)

        # 分离船只和尾迹损失
        ship_loss = losses.get('ship_loss', torch.tensor(0.0).to(device))
        wake_loss = losses.get('wake_loss', torch.tensor(0.0).to(device))

        loss_dict = {'ship': ship_loss, 'wake': wake_loss}

        # DSO 优化步骤
        dso_stats = dso_optimizer.step(loss_dict)

        total_loss += dso_stats['total_loss']
        pbar.set_postfix({
            'loss': f"{dso_stats['total_loss']:.4f}",

```

```

        'ship_l': f"{dso_stats['ship_lambda']:.3f}",
        'wake_l': f"{dso_stats['wake_lambda']:.3f}"
    })

    return total_loss / len(dataloader)

def main(args):
    # 加载配置
    cfg = load_config(args.config)

    # 创建模型
    model = build_model(cfg)
    model = model.to(args.device)

    # 创建 DSO 优化器
    dso_optimizer = DS0Optimizer(model, cfg['optimizer'])

    # 加载数据集
    train_dataset = SwimshipDataset(
        root_dir=cfg['data']['train_root'],
        annotation_file=cfg['data']['train_annotation'],
        transforms=build_transforms(cfg['data']['train_transforms'])
    )
    train_loader = torch.utils.data.DataLoader(
        train_dataset,
        batch_size=cfg['data']['batch_size'],
        shuffle=True,
        num_workers=cfg['data']['num_workers'],
        collate_fn=collate_fn
    )

    # 训练循环
    for epoch in range(cfg['epochs']):
        avg_loss = train_epoch(model, train_loader, dso_optimizer, args.device, epoch)
        print(f'Epoch {epoch}: Average Loss = {avg_loss:.4f}')

        # 保存检查点
        if (epoch + 1) % cfg['save_interval'] == 0:
            save_checkpoint(model, dso_optimizer, epoch,
                           f"{cfg['work_dir']}/epoch_{epoch}.pth")

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--config', required=True)
    parser.add_argument('--device', default='cuda')
    args = parser.parse_args()
    main(args)

```

3.5 快速测试脚本

```
# tools/quick_test.py
```

```
"""
快速测试脚本 - 使用 10 张图片验证数据通路
"""

import torch
from models.wakeship_det import WakeShipDet
from datasets.wakeship_dataset import SwimshipDataset
from utils.mask_visualizer import save_intermediate_outputs

def quick_test():
    # 创建小数据集
    dataset = SwimshipDataset(
        root_dir='data/swimship_mini/images',
        annotation_file='data/swimship_mini/annotations.json'
    )

    # 创建模型
    model = WakeShipDet(
        backbone='convnext_tiny',
        num_classes=2 # ship, wake
    )

    # 测试前向传播
    for i in range(min(10, len(dataset))):
        image, target = dataset[i]

        # 测试 backbone 输出
        features, masks = model.backbone(image.unsqueeze(0), return_masks=True)

        print(f"Image {i}:")
        print(f"  Feature shapes: {[f.shape for f in features]}")
        print(f"  Mask shapes: ship={masks[0]['ship'].shape}, wake={masks[0]['wake'].shape}")

        # 保存可视化
        save_intermediate_outputs(model, image, f'outputs/test_{i}')

    print("\n${\\vcenter{{\\hbox{{\\includegraphics[height=\\ht\\strutbox]{{/usr/share/docminer/emoji/2705.png}}}}}\\$ All tests passed!")

if __name__ == '__main__':
    quick_test()
```

第四部分：实施计划与检查清单

4.1 阶段性目标

阶段	目标	验收标准
Phase 1	环境搭建 + 代码框架	能运行 quick_test.py 不报错
Phase 2	数据通路验证	10 张图片训练一个 iteration

Table 5 – continued

阶段	目标	验收标准
Phase 3	蒙版可视化	输出清晰的船只/尾迹注意力图
Phase 4	全量训练	在 Swimship 上收敛
Phase 5	优化调试	mAP 达到预期目标

4.2 关键检查点

```
# 检查点 1: MoE 输出形状检查
def check_moe_output():
    moe = GridLevelMoE(256, num_experts=8, top_k=2)
    x = torch.randn(2, 256, 32, 32)
    out = moe(x)
    assert out.shape == x.shape, f"Shape mismatch: {out.shape} vs {x.shape}"
    print("$${\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/
    ↳ 2705.png}}}}$$ MoE output shape check passed")

# 检查点 2: 蒙版范围检查
def check_mask_range():
    model = WakeShipDet()
    x = torch.randn(1, 3, 512, 512)
    _, masks = model.backbone(x, return_masks=True)
    for stage, mask_dict in masks.items():
        ship_mask = mask_dict['ship']
        wake_mask = mask_dict['wake']
        assert ship_mask.min() >= 0 and ship_mask.max() <= 1, "Ship mask out of range"
        assert wake_mask.min() >= 0 and wake_mask.max() <= 1, "Wake mask out of range"
    print("$${\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/
    ↳ 2705.png}}}}$$ Mask range check passed")

# 检查点 3: DSO 学习率调整检查
def check_dso_lr_adjustment():
    dso = DSOOptimizer(model, {'lr': 0.0001, 'weight_decay': 0.05})

    # 模拟收敛快的任务
    losses_fast = {'ship': torch.tensor(0.1), 'wake': torch.tensor(1.0)}
    stats1 = dso.step(losses_fast)

    # 模拟收敛慢的任务
    losses_slow = {'ship': torch.tensor(1.0), 'wake': torch.tensor(0.1)}
    stats2 = dso.step(losses_slow)

    # 验证学习率调整方向
    assert stats1['ship_lambda'] < stats1['wake_lambda'], "Fast task should have lower lambda"
    assert stats2['ship_lambda'] > stats2['wake_lambda'], "Slow task should have higher lambda"
    print("$${\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/
    ↳ 2705.png}}}}$$ DSO LR adjustment check passed")
```

4.3 预期挑战与解决方案

挑战	解决方案
MoE 训练不稳定	使用负载均衡损失，确保专家利用率均匀
蒙版全 0 或全 1	添加初始化偏置，使用 sigmoid 激活
船只/尾迹特征混淆	增加专家分工约束，强制专家特化
显存不足	使用梯度累积，减少 batch size
DSO 发散	降低 temperature，增大 bias

总结

你的研究构想具有很强的创新性，特别是在：

- 1. **Grid-Level MoE 的单模态特化**：将多模态思路迁移到同模态不同特征目标
- 2. **交错残差设计**：根据特征密度选择不同变换，匹配船只/尾迹特性
- 3. **Dual Stream 蒙版引导**：利用几何先验实现跨目标注意力

建议在实现过程中：

- 1. 先实现基础版本（无 MoE、无 Dual Stream），确保 baseline 能跑通
- 2. 逐步添加模块，每添加一个模块验证性能变化
- 3. 重视可视化，确保蒙版能正确反映目标位置
- 4. 保持与 SM3Det 的公平对比，控制变量

祝研究顺利！